Progress Log:PT100

B.Tarun Tej , B.Rajaswanik

10-10-2026

0.1 10-10-2026

we learned about the circuit and we tested basic voltage divider circuit in the bread board we used the same circuit as in the sirs github repo we used 120 ohms in-place of 100 ohms. but the circuit wasnt working for some reason, then we found that the breadboards power rail is not working.

0.2 11-10-2026

we then rebuilt the whole ciircuit in an another breadboard , we got 20degC instead of 34degC Then we learned about Pt100 Least squares method

0.2.1 PT100

A **Pt100** is a specific type of **Resistance Temperature Detector (RTD)**—a sensor used to measure temperature accurately across a wide range (typically from $-200°C$ to $+850°C$).

The name breaks down into its fundamental definition:

- **Pt**: Platinum, the noble metal used for the sensing element.
- **100**: The sensor has an electrical resistance of $100\,\Omega$ **at** $0°\mathbf{C}$.

Working Principle

The Pt100 operates on the principle that the electrical resistance of metals increases as their temperature rises—a property known as a **Positive Temperature Coefficient (PTC)**.

1. **Atomic Vibration & Resistance**: When temperature increases, the platinum atoms inside the sensor vibrate more intensely. These lattice vibrations obstruct the flow of electrons, causing electrical resistance to increase.
2. **Predictable Change**: Platinum is selected because its resistance-to-temperature relationship is extremely stable, highly repeatable, and nearly linear over a broad range.
3. **Standardized Coefficient**: For a standard Pt100 sensor (per IEC 60751), the resistance increases by roughly $0.385\,\Omega$ for every $1°C$ increase in temperature.

Example Temperature-to-Resistance Values

- $-50°\text{C} \approx 80.31\,\Omega$
- $0°\text{C} = \mathbf{100.00\,\Omega}$
- $100°\text{C} \approx 138.51\,\Omega$
- $200°\text{C} \approx 175.86\,\Omega$

How the Measurement Is Made

Because the sensor itself only changes its electrical resistance, a controller or signal conditioner determines the temperature using the following process:

1. **Excitation**: A small, constant excitation current I (typically $\leq 1\,\text{mA}$ to prevent self-heating errors) is passed through the Pt100 element.
2. **Voltage Measurement**: The voltage drop V across the sensor is measured using Ohm's Law:
$$V = I \cdot R$$
3. **Conversion**: The controller calculates resistance:
$$R = \frac{V}{I}$$
It then converts this calculated resistance R directly into a temperature value using standard calibration equations (such as the Callendar–Van Dusen equation) or lookup tables.

Connection Configurations

Standard lead wires introduce parasitic resistance. To mitigate measurement errors, different wiring setups are used:

0.2.2 Least Squares approximation

Least squares approximation in a PT100 temperature sensor project is a mathematical method used to calibrate the sensor and convert raw analog readings into accurate temperature values.

A PT100 sensor changes its electrical resistance based on temperature. Microcontrollers read this as a changing voltage or digital count from an analog to digital converter. Due to hardware tolerances, wire resistance, and slight non linear sensor behavior, the raw readings do not perfectly align with ideal temperature values.

By collecting a set of known reference temperatures and their corresponding raw sensor readings, the least squares method calculates the best fitting linear or polynomial equation. It does this by minimizing the sum of the squared errors between the measured data points and the calculated trendline.

This resulting equation provides precise calibration coefficients specific to the hardware setup. Incorporating this optimized equation into the microcontroller firmware allows the system to convert raw sensor measurements into precise temperature readings efficiently with minimal computational effort.

0.3 12-09-2026

we started soldering the circuit on the perfboard have donr the soldering very slowly so it took us an whole day excluding classes te the soldering part

0.4 13-09-2026

WE learned about the code and its working

1. System Overview & Hardware Connections

The PT100 RTD sensor is connected in a voltage divider configuration with a $100\,\Omega$ resistor (R_1) to measure analog voltage on pin A0 of the Arduino UNO[cite: 1, 2]. The temperature output is displayed on a 16×2 character LCD operating in 4-bit mode[cite: 1].

```
#include <LiquidCrystal.h>
```

Includes the LiquidCrystal library, which provides functions to control character LCD modules.

```
const int RS_PIN = 7;
const int EN_PIN = 6;
const int DB4_PIN = 5;
const int DB5_PIN = 4;
const int DB6_PIN = 3;
const int DB7_PIN = 2;
```

Defines the Arduino digital pins connected to the LCD control and data lines (RS to pin 7, Enable to pin 6, and DB4 through DB7 to pins 5, 4, 3, and 2)[cite: 1].

```
LiquidCrystal lcd(RS_PIN, EN_PIN, DB4_PIN, DB5_PIN, DB6_PIN, DB7_PIN);
```

Creates the LCD controller object named `lcd` using the assigned digital pins[cite: 1].

```
const int SENSOR_PIN = A0;
```

Sets analog input pin A0 (Arduino physical pin 9) as the sensor reading pin connected to the voltage divider output node[cite: 1, 2].

```
const float R1 = 100.0;
```

Sets the reference resistor R1 value to 100.0 ohms[cite: 2].

```
const int N = 6;
```

Defines the total number of calibration data points as 6.

```
const float R_data[N] = {100.00, 107.79, 115.54, 123.24, 130.89, 138.51};
```

Creates an array storing standard PT100 resistance values in ohms corresponding to known reference temperatures.

```
const float T_data[N] = {0.0, 20.0, 40.0, 60.0, 80.0, 100.0};
```
Creates an array storing the corresponding known temperatures in degrees Celsius.

```
float slope_m = 0.0;
float intercept_c = 0.0;
```
Declares global variables to store the calculated slope (m) and y-intercept (c) of the fitted line.

```
void calculateLeastSquares(const float x[], const float y[], int n, float &m,
float &c) {
```
Declares the function that performs Ordinary Least Squares regression, taking input arrays x and y, array length n, and outputting slope m and intercept c by reference.

```
float sum_x = 0.0, sum_y = 0.0, sum_xy = 0.0, sum_x2 = 0.0;
```
Initializes variables to accumulate the sum of x, sum of y, sum of $x \cdot y$ products, and sum of x^2.

```
for (int i = 0; i < n; i++) {
  sum_x += x[i];
  sum_y += y[i];
  sum_xy += x[i] * y[i];
  sum_x2 += x[i] * x[i];
}
```
Iterates through all calibration points to calculate total sums needed for linear regression.

```
m = (n * sum_xy - sum_x * sum_y) / (n * sum_x2 - sum_x * sum_x);
```
Calculates the optimal line slope m using the Ordinary Least Squares formula.

```
c = (sum_y - m * sum_x) / n;
```
Calculates the y-intercept c using the derived slope and mean value differences.

```
}
```
Ends the `calculateLeastSquares` function definition.

```
void setup() {
```
Begins the setup configuration routine that runs once at power up.

```
lcd.begin(16, 2);
```
Initializes the LCD screen hardware to operate with 16 columns and 2 rows.

```
calculateLeastSquares(R_data, T_data, N, slope_m, intercept_c);
```
Calls the Least Squares function on startup to compute `slope_m` and `intercept_c` from the reference dataset.

```
lcd.print("PT100 Calibrated");
lcd.setCursor(0, 1);
lcd.print("Least Squares OK");
delay(1500);
lcd.clear();
```
Displays calibration confirmation text on the LCD screen for 1.5 seconds, then clears the display screen.

```
}
```
Ends the `setup` function.

```
void loop() {
```
Begins the main execution loop that runs continuously.

```
int adcValue = analogRead(SENSOR_PIN);
```
Reads the raw 10-bit analog-to-digital converter value (ranging from 0 to 1023) at pin A0[cite: 1, 2].

```
if (adcValue > 0 && adcValue < 1023) {
```
Validates that the ADC reading is within a realistic open/short circuit range to prevent division-by-zero errors.

```
float R_pt100 = R1 * ((1023.0 / (float)adcValue) - 1.0);
```
Calculates the PT100 sensor resistance in ohms by rearranging the voltage divider circuit equation[cite: 1, 2].

```
float temperature = (slope_m * R_pt100) + intercept_c;
```
Calculates temperature in degrees Celsius using the calibrated linear equation $T = m \cdot R + c$.

```
lcd.setCursor(0, 0);
lcd.print("R: ");
lcd.print(R_pt100, 2);
lcd.print(" Ohm ");
```
Positions the display cursor at row 1, column 1, and prints the measured PT100 resistance formatted to 2 decimal places.

```
lcd.setCursor(0, 1);
lcd.print("Temp:  ");
lcd.print(temperature, 2);
lcd.print(" ");
lcd.write(223);
lcd.print("C ");
```
Positions the display cursor at row 2, column 1, prints the computed temperature for-

matted to 2 decimal places, outputs character 223 for the degree symbol, and appends the letter C.

```
} else {
  lcd.setCursor(0, 0);
  lcd.print("Sensor Error ");
}
```
Displays an error message on line 1 if the sensor circuit reading falls outside expected voltage boundaries.

```
delay(500);
```
Pauses execution for 500 milliseconds before taking the next sensor reading.

```
}
```
Ends the main loop function.

0.5 14-09-2026

we took measurements and started designing the 3d module and started the decade counter project

0.6 15-09-2026

we completed the 3d module designing